

Lab 3: Building Your First API with FastAPI

Overall Objectives:

- Students will understand the basic role of a server-side API.
- Students will be able to install FastAPI and Uvicorn.
- Students will create a minimal FastAPI application with a few simple endpoints.
- Students will run their FastAPI application using Uvicorn.
- Students will test their API endpoints using a web browser and FastAPI's automatic interactive documentation.

Instructions: Building Your First API with FastAPI

Part 1: Setup

1. Open Your Project:

- Ensure you are in your VS Code window, connected to the "Dev Container: Python 3" for your `Day2_Python_Labs` project.
- Open the integrated terminal (`Terminal > New Terminal`).

2. Install FastAPI and Uvicorn:

- In the integrated terminal (inside the Dev Container), run the following command to install FastAPI and Uvicorn (which includes standard server features):

```
pip install "fastapi[all]" uvicorn[standard]
```

- `"fastapi[all]"` installs FastAPI along with optional dependencies like `pydantic` for data validation and `python-multipart` for form data, which are generally useful.
- `uvicorn[standard]` installs Uvicorn with standard, high-performance components.
- Verify the installation (optional): `pip list` should now show `fastapi` and `uvicorn` among the installed packages.

3. Create Your API File:

- In the VS Code Explorer, create a new Python file in your `Day2_Python_Labs` folder. Name it `main_api.py`. This will be where your API code lives.

Part 2: Creating Your First Endpoint

1. Import FastAPI:

- Open `main_api.py` and add the following line at the top:

```
from fastapi import FastAPI
```

2. Create a FastAPI "App" Instance:

- Below the import, create an instance of the FastAPI class. This `app` object will be the main point of interaction for creating all your API "routes" or "endpoints."

```
app = FastAPI()
```

3. Define Your First "Path Operation" (Root Endpoint):

- We'll create an endpoint that responds to HTTP GET requests at the root path (`/`).
- FastAPI uses decorators to associate a URL path and HTTP method with a Python function.
- Add the following code:

```
@app.get("/")
async def read_root():
    return {"message": "Hello from my First FastAPI API!"}
```

- Explanation:
 - `@app.get("/")` : This is a decorator.
 - `@app` : Tells FastAPI this is for our `app` instance.
 - `.get` : Specifies that this function will handle HTTP GET requests.
 - `("/")` : Specifies the URL path for this endpoint (the root path).
 - `async def read_root():` : This defines an asynchronous function. FastAPI is built for asynchronous programming (which is great for I/O-bound tasks like web requests), so `async def` is common. For now, just know this is how FastAPI prefers its endpoint functions.
 - `return {"message": "Hello from my First FastAPI API!"}` : FastAPI will automatically convert this Python dictionary into a JSON response that gets sent back to the client.

Part 3: Running Your API with Uvicorn

1. The `uvicorn` Command:

- Go to your integrated VS Code terminal. Make sure you are in the `Day2_Python_Labs` directory where `main_api.py` is saved.
- To run your API, type the following command:

```
uvicorn main_api:app --reload
```

- Explanation:
 - `uvicorn` : The command to start the Uvicorn server.
 - `main_api` : This refers to your Python file `main_api.py`.
 - `:app` : This refers to the `app = FastAPI()` instance you created inside `main_api.py`.
 - `--reload` : This is a very useful development feature. It tells Uvicorn to automatically restart the server whenever you save changes to your `main_api.py` file.

2. Observe the Output:

- You should see output similar to this:

```
INFO:      Will watch for changes in ['./*']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [xxxxx]
INFO:      Started server process [yyyyy]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

- This means your API server is now running and listening for requests on `http://127.0.0.1:8000`. `127.0.0.1` is `localhost` (your own machine), and `8000` is the default port Uvicorn uses.

Part 4: Testing Your API (25 minutes)

1. Test with a Web Browser:

- Open your web browser (Chrome, Firefox, Edge, etc.).
- Go to the address: `http://127.0.0.1:8000`
- You should see the JSON response from your `read_root` function:

```
{"message": "Hello from my First FastAPI API!"}
```

2. Explore the Automatic Interactive API Documentation (Swagger UI):

- This is one of FastAPI's best features!
- In your web browser, go to: `http://127.0.0.1:8000/docs`
- You'll see a beautiful, interactive API documentation page.
- You should see your `/` endpoint listed.
- Click on it to expand it.
- Click the "Try it out" button.
- Click the "Execute" button.

- Scroll down, and you'll see:
 - The `curl` command you could use to test it from a terminal.
 - The "Request URL".
 - The "Server response" including the Code (e.g., `200`), the response body (your JSON message), and response headers.
 - This `/docs` page is automatically generated by FastAPI from your Python code and type hints!
3. Explore the Alternative Automatic Documentation (ReDoc):
- FastAPI also provides another documentation style.
 - In your web browser, go to: `http://127.0.0.1:8000/redoc`
 - This shows a different view of your API documentation.

Part 5: Adding an Endpoint with a Path Parameter (30 minutes)

Let's make your API a bit more dynamic by accepting a parameter in the URL path.

1. Stop Uvicorn (if needed):
 - If Uvicorn is still running in your terminal, press `ctrl+c` to stop it. (If you used `--reload`, it might restart automatically after you save the file, but it's good practice to know how to stop/start).
2. Define the New Path Operation:
 - In `main_api.py`, add the following new function *below* your `read_root` function (but still part of the `app`):

```
@app.get("/greet/{name}")
async def greet_by_name(name: str):
    return {"greeting": f"Hello, {name}! Welcome to my API."}
```

- Explanation:
 - `@app.get("/greet/{name}")`:
 - The path is now `/greet/{name}`.
 - `{name}` is a path parameter. FastAPI understands that this part of the URL is a variable.
 - `async def greet_by_name(name: str):`:
 - The function now accepts an argument `name`.
 - `name: str`: This is a Python type hint. It tells FastAPI (and other developers, and tools like linters) that the `name` parameter is expected to be a string. FastAPI uses this for data validation and documentation.
 - `return {"greeting": f"Hello, {name}! Welcome to my API."}`: We use an f-string to include the `name` path parameter in the response.

3. Run/Restart Uvicorn:

- If you stopped Uvicorn, restart it:

```
uvicorn main_api:app --reload
```

- If it was already running with `--reload`, it should have detected the change and restarted automatically when you saved `main_api.py`. Check the terminal output.

4. Test the New Endpoint:

- From your browser:
 - Go to `http://127.0.0.1:8000/greet/YourName` (replace `YourName` with your actual name or any string).
 - You should see: `{"greeting": "Hello, YourName! Welcome to my API."}`
 - Try a different name: `http://127.0.0.1:8000/greet/Student`
- From the `/docs` page:
 - Refresh `http://127.0.0.1:8000/docs`.
 - You should now see your new `/greet/{name}` endpoint listed.
 - Expand it, click "Try it out".
 - You'll see a field where you can enter a value for the `name` parameter.
 - Enter a name and click "Execute". Observe the response.

Part 6: (Bonus Stretch Goal) Endpoint with a Query Parameter (15 minutes, if time permits)

Query parameters are another way to pass data to an API, typically for filtering or optional settings. They appear after a `?` in the URL.

1. Define the New Path Operation:

- In `main_api.py`, add:

```
@app.get("/items/")
async def read_item(item_id: int, query_param: str | None = None):
    response_data = {"item_id": item_id}
    if query_param:
        response_data["query_param_received"] = query_param
    return response_data
```

- Explanation:
 - `@app.get("/items/")`: A new endpoint.
 - `async def read_item(item_id: int, query_param: str | None = None):`
 - `item_id: int`: This is now a required query parameter because it doesn't have a default value. FastAPI will expect it in the URL like `?`

`item_id=123` .

- `query_param: str | None = None` : This is an optional query parameter.
 - `str | None` : The type hint means it can be a string OR `None` .
 - `= None` : This provides a default value. If the client doesn't provide `query_param` in the URL, its value inside the function will be `None` .
- The function logic checks if `query_param` was provided and includes it in the response if it was.

2. Run/Restart Uvicorn and Test:

- Save `main_api.py` . Uvicorn (with `--reload`) should restart.
- Test from browser/docs:
 - `http://127.0.0.1:8000/items/?item_id=10`
 - Expected: `{"item_id":10}`
 - `http://127.0.0.1:8000/items/?item_id=20&query_param=test_value`
 - Expected: `{"item_id":20, "query_param_received":"test_value"}`
 - `http://127.0.0.1:8000/items/` (without `item_id`)
 - FastAPI will return a validation error because `item_id` is required! Check the `/docs` page to see how it indicates required parameters.
 - Explore this new endpoint in `/docs` . Notice how it shows fields for both `item_id` (required) and `query_param` (optional).